Practical Application of Agile PM Principles – Case Study

Rohayanti Hassan
Faculty of Computing
Universiti Teknologi Malaysia

This slide contains material that is copied from the book Agile Project Manager_v2_handbook by Agile Business Consortium. All rights reserved. No part of this material may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright owner. This material is used here for educational purposes under the doctrine of fair use, with the intention to support learning and teaching.

For more information, please refer to the original source:
Book Title: Agile Project Manager_v2_handbook
Author: Agile Business Consortium
Publisher: Buckland Media Group Limited
Year of Publication: 2017
Please consult the book for further details and complete content.

Outline

- User Stories
- DSDM Practice : MoSCoW Prioritization
  - Introduction
  - The MoSCoW Rules
  - MoSCow relating to a Specific Timeframe

- DSDM Practice : Timeboxing
  - Timeboxing Options
  - The Daily Stand Up (Scrum)
  - Timeboxes - The Wider Context

DSDM Practice : MoSCoW Prioritisation

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

The Case Study

Case Study: Attendance Management System Using Facial Recognition (AMS-FR)

The Faculty of Computing plans to digitalize attendance monitoring for lectures, tutorials, and labs by implementing an **Attendance Management System using Facial Recognition (AMS-FR)**. The system replaces manual sign-in sheets with automated face detection and recognition at classroom entrances or via in-class capture devices.

When students enter the classroom, a camera captures real-time facial images and matches them against the Faculty's student database. Recognized students are instantly marked as *present*, while mismatches or unverified faces are logged for lecturer review. The system also supports late arrival tagging, absentee alerts, and generates analytics such as attendance trends, weekly attendance summaries, low-attendance warnings, and course-level dashboards.

The system supports:

- **Students** → seamless attendance capture
- **Lecturers** → automated registers, review mismatches, generate evidence
- **Administrators** → analytics, reporting, system configuration

User Story #1

Automated Facial Recognition Attendance Capture

As a Lecturer
I want the system to automatically record student attendance using facial recognition at classroom entry
So that attendance is captured accurately and without manual effort.

Functional Requirements

System must capture real-time images from the classroom camera.
System must match detected faces with registered student profiles.
System must auto-tag attendance as “Present”, “Late”, or “Unverified”.
System must allow the lecturer to review and resolve unverified attendance.
System must store attendance records in the database.

Non-Functional Requirements

Recognition accuracy ≥ 90% in normal classroom lighting.
Response time ≤ 3 seconds per face match.
System uptime ≥ 99% during lecture hours.
Data must comply with university privacy and security policies (encrypted storage).
Camera feed must handle at least 20 simultaneous face detections.

Acceptance Criteria

Given a student walks into class, when the system detects their face, then the system must match it to a valid student profile.
When a match succeeds, the system must mark the student as *Present* with a timestamp.
When a match fails, the system must mark the entry as *Unverified* and place it into the lecturer’s review panel.
The attendance status must be stored in the database and appear correctly on the lecturer dashboard.

User Story #1

Automated Facial Recognition Attendance Capture

Tasks & Subtasks

1. UI / UX Design

Design attendance dashboard for lecturers
Design unverified-attendance review screen
Design live camera feed monitoring UI

2. Backend Logic

Implement facial detection and recognition API
Implement attendance tagging logic
Build database schema for attendance records
Develop mismatch review service

3. Integration

Connect camera hardware feed to backend
Integrate facial recognition model with API
Implement authentication for lecturers

4. Testing

Unit testing facial recognition module
System testing for real-time capture
Performance test under multiple faces
Security testing on data privacy

5. Deployment

Deploy backend to server environment
Configure classroom devices (camera + client app)
Monitor logs and optimize model performance

User Story #2

Attendance Analytics Dashboard

As an Administrator
I want the system to generate attendance analytics and visual reports
So that I can monitor trends, detect low attendance patterns, and support academic decisions.

Functional Requirements

System must compute weekly and monthly attendance summaries.
System must display analytics (charts, heatmaps, trend lines).
System must allow export of analytics (PDF/CSV).
System must generate low-attendance alerts.
System must support filtering by class, lecturer, semester, and program.

Non-Functional Requirements

Dashboard must load analytics within 5 seconds.
Charts must remain readable on desktop and mobile layouts.
Reports must support export formats ≤ 5MB per file.
System must maintain data integrity with accurate calculations.
High availability during peak reporting periods.

Acceptance Criteria

The dashboard must show accurate attendance percentages for all selected classes.
When an administrator filters by semester/program, the data must update correctly.
When attendance falls below the configurable threshold, the system must generate an alert.
Exported analytics must match what is shown on screen.

User Story #2

Attendance Analytics Dashboard

Tasks & Subtasks

1. UI / UX Design

Design analytics dashboard layout
Design charts (bar, line, pie, heatmap)
Design export report interface

2. Backend Logic

Build analytics computation engine
Implement filtering and dataset aggregation
Implement alert generation service

3. Integration

Connect dashboard frontend with analytics API
Integrate report export module
Sync attendance database to ensure consistent data

4. Testing

Validate analytics calculations
Test filters and chart rendering
Test export functionality
Load testing for large datasets

5. Deployment

Deploy analytics microservice
Configure cron jobs for data aggregation
Set up monitoring for performance and errors

DSDM Practice - MoSCoW Prioritisation

The project team must analyse functional and non-functional requirements, prioritise them using MoSCoW classification.

Must have = Minimum Usable SubseT (Guaranteed)
Without this requirement . no point on this date or not legal or unsafe or not a viable solution

Should have
Important but not vital
May be painful to leave out but solution still viable
May need workaround, which may be temporary

Could have
Wanted or desirable but less important
Less impact if left out (compared with a Should)

Won't have this time
Will not be delivered in this timeframe



Functional Requirements (FR) – MoSCoW Prioritisation

M — Must Have (Essential for system operation)

These requirements are critical for the system to function. Without them, the system fails its core purpose.

1. **FR1**: Capture real-time images from the classroom camera for facial detection.
2. **FR2**: Match detected faces with registered student profiles using facial recognition.
3. **FR3**: Automatically mark student attendance as *Present*, *Late*, or *Unverified*.
4. **FR4**: Store attendance records securely in the database.
5. **FR5**: Provide lecturers with a dashboard to view real-time attendance.

S — Should Have (Important but not immediately mandatory)

These add significant value but are not critical for initial deployment.

6. **FR6**: Allow lecturers to review and resolve unverified or mismatched attendance.
7. **FR7**: Generate attendance analytics (weekly/monthly summaries).
8. **FR8**: Support filtering (course, section, semester).
9. **FR9**: Allow export of attendance reports (CSV/PDF).

C — Could Have (Desirable but lower importance)

Enhancements that improve user experience but are not essential.

10. **FR10**: Provide mobile-responsive dashboards.
11. **FR11**: Integrate with student mobile app for notifications (absent alerts).
12. **FR12**: Provide real-time heatmap visualisation of attendance trends.

W — Won't Have (Not planned for current release)

Listed to prevent scope creep.

13. **FR13**: Integration with biometric devices other than facial recognition (e.g., fingerprints).
14. **FR14**: Inter-faculty integration for a university-wide attendance system (future phase).

Non-functional Requirements (FR) – MoSCoW Prioritisation

M — Must Have

1. **NFR1 (Performance)**: Facial recognition response time ≤ 3 seconds per person.
2. **NFR2 (Accuracy)**: Facial recognition accuracy ≥ 90%.
3. **NFR3 (Security)**: All personal data must be encrypted (at rest + in transit).
4. **NFR4 (Availability)**: System uptime ≥ 99% during lecture periods.

S — Should Have

5. **NFR5 (Scalability)**: System should support at least 20–30 face detections simultaneously.
6. **NFR6 (Usability)**: Interface should follow faculty UI standards and be lecturer-friendly.
7. **NFR7 (Compatibility)**: System should run on standard classroom hardware (CCTV/IP camera + laptop).

C — Could Have

8. **NFR8 (Portability)**: Admin dashboard could support tablets for mobile monitoring.
9. **NFR9 (Maintainability)**: System could auto-generate system health logs for maintenance.

W — Won't Have

10. **NFR10**: Cloud auto-scaling architecture for large multi-campus operations (future enhancement).

Exercise #1: Consolidated (FR + NFR) MoSCoW Table

Req ID	Requirements	Moscow Classification	Justification

Requirement	Description	MoSCoW Priority
FR1	Real-time image capture	Must
FR2	Facial recognition matching	Must

*** User Microsoft Excel**

Exercise #2: Add on with Tasks and Subtasks

Req ID	Requirements	Moscow Classification	Task	Sub Task

Req ID	Requirement Description	Priority (MoSCoW)	Tasks	Sub-Tasks
FR1	Real-time image capture from camera	Must	Develop camera integration module	• Connect IP/CCTV camera feed • Implement continuous frame capture • Error handling for camera offline

*** User Microsoft Excel**

DSDM Practice : Timeboxing

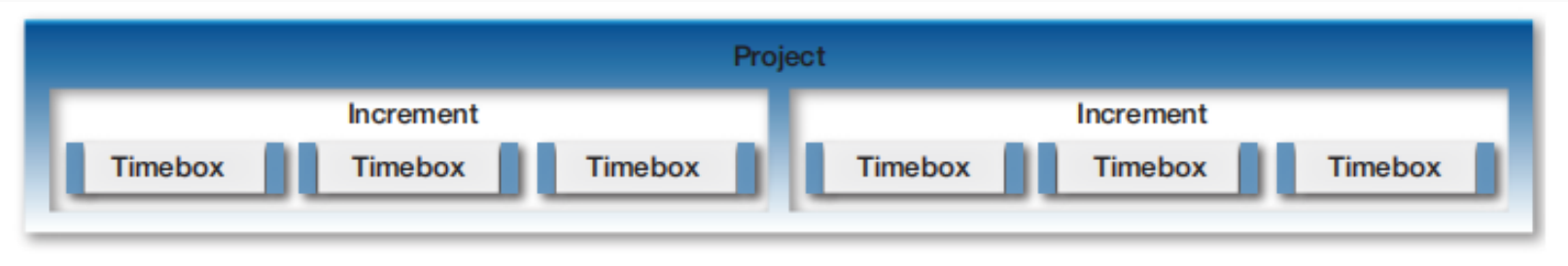
UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

Timeboxes - How Long?

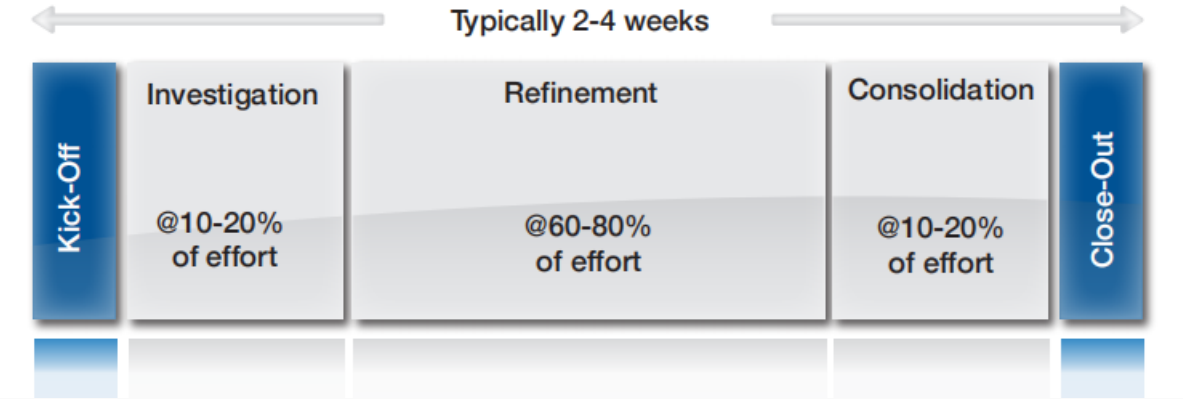
During Foundations, Agile PM helps Solution Development Team agree preferred length of Timebox

- Short enough to keep team focused
- Long enough to achieve something complete and meaningful
- Usually between 2 and 4 weeks
- Longer Timeboxes should be avoided unless really necessary



Timeboxing - Which style?

- DSDM Structured Timebox



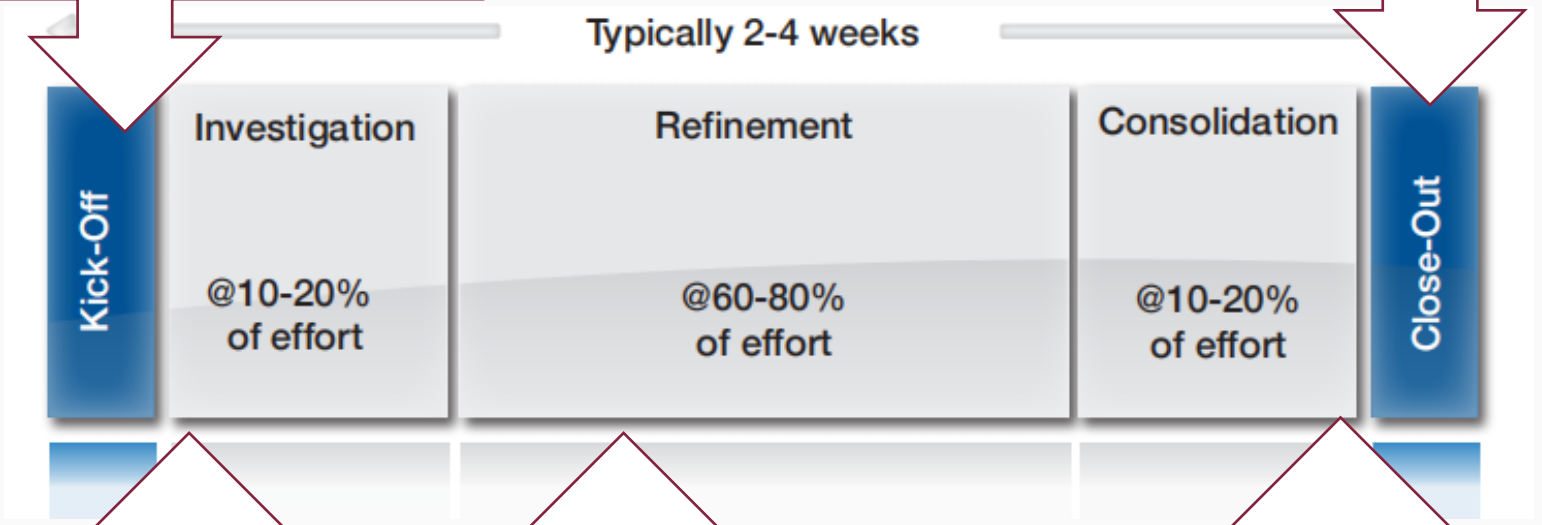
- Free Format Timebox



A DSDM Structured Timebox

- Check TB objectives in Delivery Plan still feasible
- Agree acceptance criteria, where possible
- Review availability of SDT members
- Highlight known dependencies

- Sign-off what has been delivered
- Assess impact of what has not been "done"
- Build on good experience
- Avoid repeating same mistakes
- Define issues to be resolved for next TB



- Understand detail and priorities of work to be done
- Review dependencies, Timebox Plan and risks

- Work on the Solution in line with agreed MoSCoW priorities
- Review at end of Refinement (work should be nearly ready)

- Complete actions from Refinement review
- Complete final testing
- Ensure organisational or project standards have been met, with any final quality checks
- Review at end of Consolidation, ensuring TB objectives are met.
- Any products not meeting acceptance criteria deemed as "Not Delivered"

Timebox Plan

There are two Solution Development Teams, and each team is assigned one user story. The project duration is one month, and each sprint lasts two weeks. Please prepare a detailed timebox plan for both Solution Development Teams.

⭐ Overview

Team	User Story	Focus Area
Team A	User Story 1	Automated Facial Recognition Attendance Capture
Team B	User Story 2	Attendance Analytics Dashboard

Project Timeline: **1 Month = 4 Weeks = 2 Sprints**

🟦 TIMEBOX PLAN – SPRINT 1 (Weeks 1–2)

🎯 Sprint 1 Objective

- Establish core system foundations for each user story.
- Deliver initial working components for recognition and analytics.

TB 1: TEAM A – User Story 1: Automated Attendance Capture

INVESTIGATE (Days 1–3)

- Analyse classroom camera requirements
- Validate facial recognition model feasibility
- Identify attendance states (Present, Late, Unverified)
- Draft initial database schema for attendance records

REFINE (Days 4–10)

- Integrate camera feed (live capture)
- Develop facial detection & matching prototype
- Implement attendance tagging logic (version 1)
- Create basic lecturer dashboard for viewing attendance

CONSOLIDATE (Days 11–14)

- Debug issues in face detection
- Improve recognition threshold accuracy
- Conduct initial classroom testing
- Deploy Sprint 1 build to staging & conduct sprint review

Sprint 1 Deliverables (Team A)

- ✔ Camera integration
- ✔ Basic facial detection engine
- ✔ Attendance tagging v1
- ✔ Basic dashboard view

TB 1: TEAM B – User Story 2: Attendance Analytics Dashboard

INVESTIGATE (Days 1–3)

- Define analytics requirements: weekly %, monthly %, trend data
- Draft dashboard wireframes
- Confirm data structure and filtering fields (course/section/semester)

REFINE (Days 4–10)

- Develop analytics computation engine (version 1)
- Build chart prototypes (bar chart, line chart)
- Implement filtering backend (course/section/semester)
- Create initial dashboard layout

CONSOLIDATE (Days 11–14)

- Validate accuracy of analytics data
- Fix UI layout and chart rendering issues
- Improve dashboard responsiveness
- Deploy Sprint 1 build to staging & conduct sprint review

Sprint 1 Deliverables (Team B)

- ✔ Analytics engine v1
- ✔ Filters implemented
- ✔ Chart prototypes
- ✔ Basic dashboard shell

TB 2: TEAM A – User Story 1: Automated Attendance Capture

INVESTIGATE (Days 1–3)

- Review Sprint 1 issues: accuracy gaps, performance delays
- Analyse multi-face detection scenarios
- Identify security requirements (encryption, HTTPS)

REFINE (Days 4–10)

- Develop "Unverified Attendance Review Panel"
- Implement manual verification & correction options
- Improve performance to ≤ 3 seconds per recognition
- Enhance dashboard with real-time updates
- Integrate secure API connections

CONSOLIDATE (Days 11–14)

- Conduct load testing (20–30 faces simultaneously)
- Achieve ≥ 90% facial recognition accuracy
- Final system integration testing
- Sprint review & production deployment

Sprint 2 Deliverables (Team A)

- ✔ Full facial recognition workflow
- ✔ Unverified attendance review panel
- ✔ Optimised dashboard
- ✔ Performance & accuracy improvements

TB: TEAM B – User Story 2: Attendance Analytics Dashboard

INVESTIGATE (Days 1–3)

- Finalize export formats (PDF, CSV)
- Define low attendance alert rules (thresholds)
- Identify heatmap visual design

REFINE (Days 4–10)

- Implement export module (PDF/CSV)
- Build weekly/monthly summary report generator
- Develop low attendance alert module
- Create attendance heatmap visualization
- Improve UX for dashboard navigation

CONSOLIDATE (Days 11–14)

- Correct export formatting errors
- Validate consistency of analytics and reports
- Optimize UI responsiveness (desktop/tablet/mobile)
- Sprint review & deploy final version to production

Sprint 2 Deliverables (Team B)

- ✔ Full analytics dashboard
- ✔ Export (PDF/CSV) working
- ✔ Low-attendance alert engine
- ✔ Heatmap analytics
- ✔ Fully responsive UI

Exercise #3: Generate Gantt chart for Timebox Plan

Using Microsoft Excel, generate a Gantt chart for the timebox plan for each team.

Include all activities and the key milestones to be delivered.

Exercise #4: Project Extension

✅ 1. New Additional User Story (User Story 3)

User Story 3 – Student Self-Service Attendance & Notification Portal

As a Student

I want to view my personal attendance history and receive automated notifications

So that I can monitor my attendance status and take corrective actions when necessary.

Functional Requirements

- FR15: Students can log in to view daily, weekly, and semester attendance.
- FR16: System sends automated notifications for low attendance warnings.
- FR17: Students can download their attendance summary (PDF).
- FR18: Students can request correction if wrongly marked as absent (optional future enhancement).

Non-Functional Requirements

- NFR11: Dashboard must load within 5 seconds.
- NFR12: Notification delivery rate ≥ 98%.
- NFR13: Student data must remain private and secured by access control (role-based).
- NFR14: Interface must be mobile-responsive for smartphones.

Acceptance Criteria

- Students can view all attendance records for the semester with accurate percentages.
- When attendance drops below the configured threshold, the system triggers a notification.
- PDF downloads must match attendance displayed in the system.
- Only authenticated students may access their own data.

The project is extended for one additional Sprint (Sprint 3) to incorporate a new user story involving a Student Self-Service Attendance & Notification Portal. This extension aims to enable students to view their attendance, receive automated low-attendance notifications, and download attendance summaries. Please prepare an **additional timebox plan for Sprint 3** to accommodate the new user story and deliver the required features within a two-week sprint.



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my